



GA4-220501095-AA2-EV05

DESARROLLAR LA ARQUITECTURA DE SOFTWARE,  
DE ACUERDO CON EL PATRÓN DE DISEÑO SELECCIONADO

July Andrea Herreño  
Edwin Ramírez Rodríguez

Servicio Nacional de Aprendizaje – SENA

Análisis y Desarrollo de Software

Instructor:

Jorge Mario Hernández Torres

Bogotá Distrito Capital  
03 de diciembre de 2025

## **INTRODUCCIÓN**

El presente trabajo tiene como objetivo desarrollar la arquitectura de software para una aplicación web y móvil orientada a la gestión de finanzas personales. Esta solución permitirá a los usuarios registrar ingresos, egresos y calcular la amortización de sus deudas, ofreciendo una experiencia intuitiva y segura. Para lograrlo, se emplearán diagramas UML que faciliten la comprensión del flujo del sistema, su estructura y la interacción entre componentes.

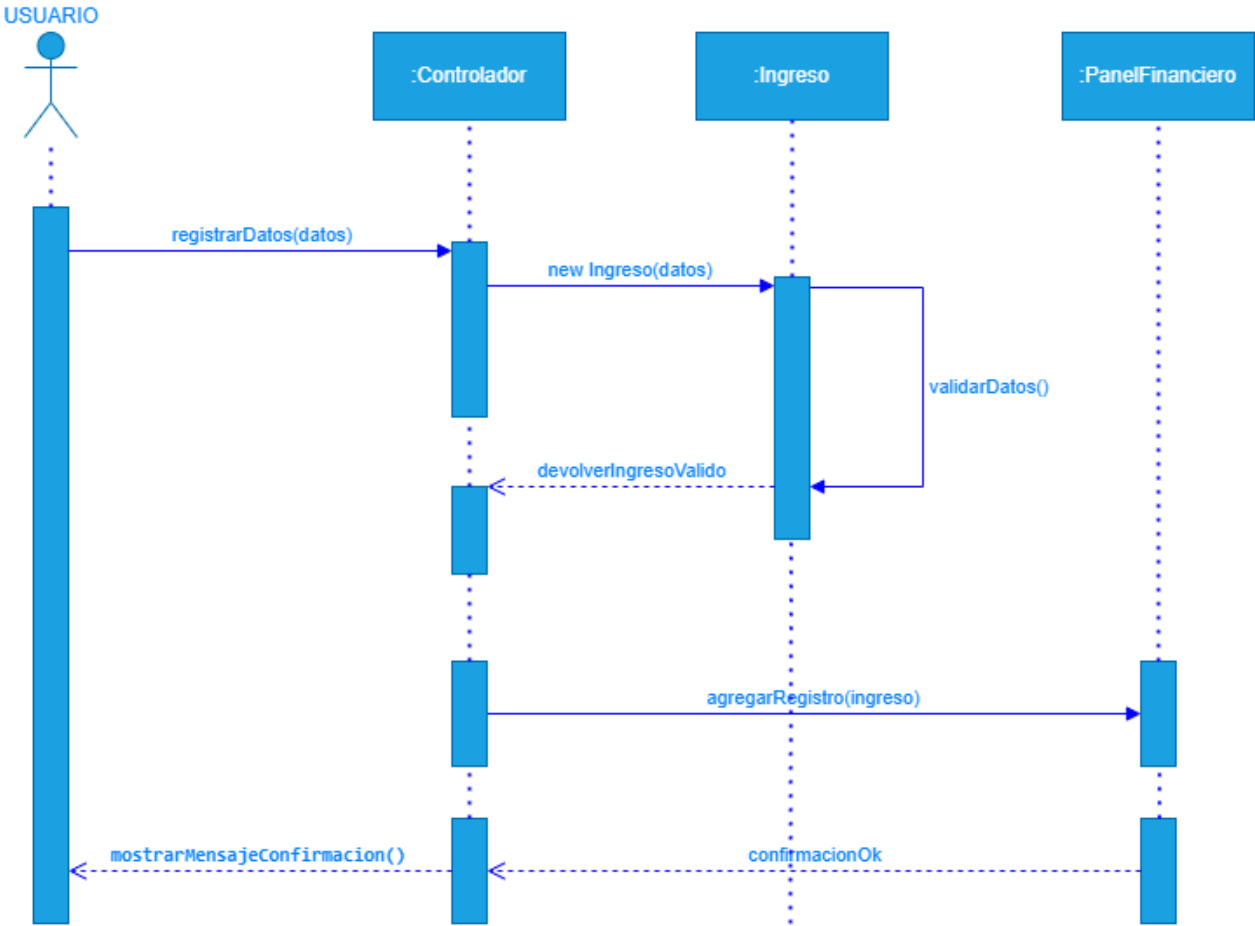
## PROBLEMA

Se requiere diseñar una aplicación web y móvil por medio de la cual un usuario pueda llevar sus finanzas personales por lo cual será necesario que el usuario cuente con una serie de funcionalidades en dentro de su cuenta para poder interactuar con su información financiera y llevar un control de sus gastos, deudas o la amortización de sus créditos. Para ello se requiere que usuario pueda interactuar con el la aplicación de acuerdo a los siguientes parámetros:

- a. El usuario deberá poder contar con acceso a un autenticador de usuario por medio del cual se pueda registrar y acceder a la aplicación web o móvil.
- b. Una vez ingrese, el sistema le deberá mostrar por pantalla un mensaje de bienvenida.
- c. El sistema le deberá mostrar por pantalla al usuario un mensaje que indique “Inicia tu control financiero” luego mostrará el botón iniciar.
- d. Cuando el usuario selecciona la opción iniciar se desplegará un formulario cuya cabecera deber anunciar al usuario que introduzca sus datos de ingresos mensuales, una vez estén introducidos los datos que el formulario pida, el usuario deberá finalizar y los datos ingresados deberán ser almacenados en la base de datos.
- e. Una vez finalizado el paso anterior el sistema deberá desplegar un nuevo formulario parecido al anterior, pero con la diferencia de que en este caso él deberá poder ingresar los datos de egresos mensuales, una vez finaliza el sistema debe almacenar la información en la base de datos.
- f. Una vez el usuario ha introducido sus datos de ingresos y egresos, el sistema deberá mostrar por pantalla el resumen de ingresos y egresos y deberá mostrar un botón que indique “Calcula tu deuda”.
- g. Si el usuario selecciona la opción “Calcula tu deuda”, el sistema deberá desplegar un formulario por medio del cual se le pedirá al usuario que introduzca los datos de la deuda como el monto, la tasa de interés, el plazo, etc., luego al finalizar de diligenciar los campos el sistema habilita un botón que diga “Generar amortización” luego se mostrará por pantalla el cuadro de amortización con un botón que indique guardar registros.
- h. Si el usuario guarda los registros estos serán almacenados en la base de datos, si el usuario sale de la ventana sin guardar el sistema le pedirá de nuevo que, si desea guardar los cambios, en caso de que no, ningún dato será almacenado.

DIAGRAMAS DE SECUENCIA (DS)

DS1:



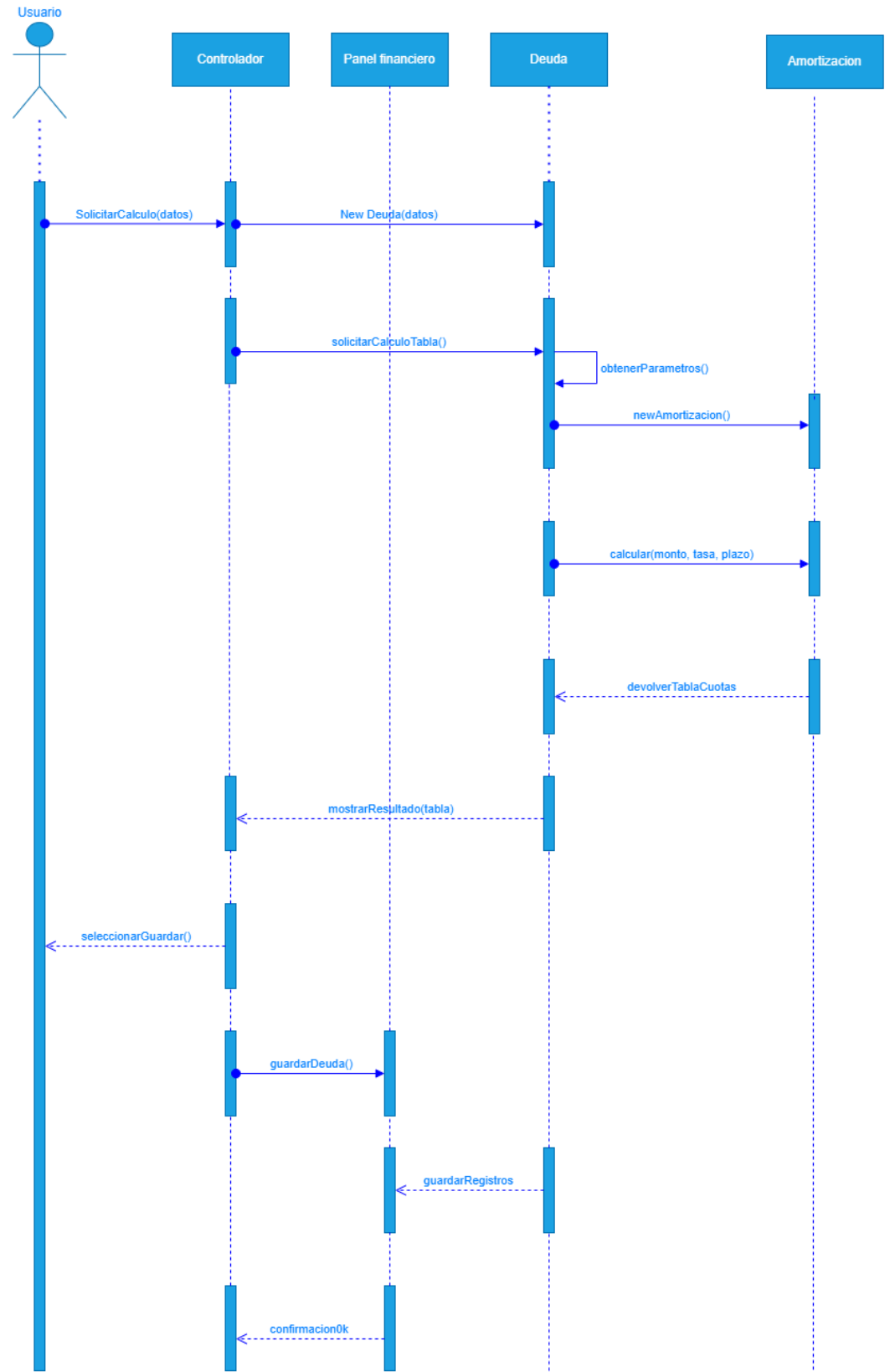
Este diagrama representa el flujo inicial cuando el usuario accede a la aplicación. Incluye:

- Autenticación del usuario: registro e inicio de sesión.
- Interacción inicial: mensaje de bienvenida y opción para iniciar el control financiero.
- Formulario de ingresos: captura y almacenamiento en la base de datos.

Análisis:

El diagrama muestra una secuencia clara y lineal, lo que facilita la comprensión del proceso. Sin embargo, se debe considerar la validación de datos antes de almacenarlos, para evitar inconsistencias. También es importante definir mensajes de error en caso de fallos en la autenticación.

DS2:



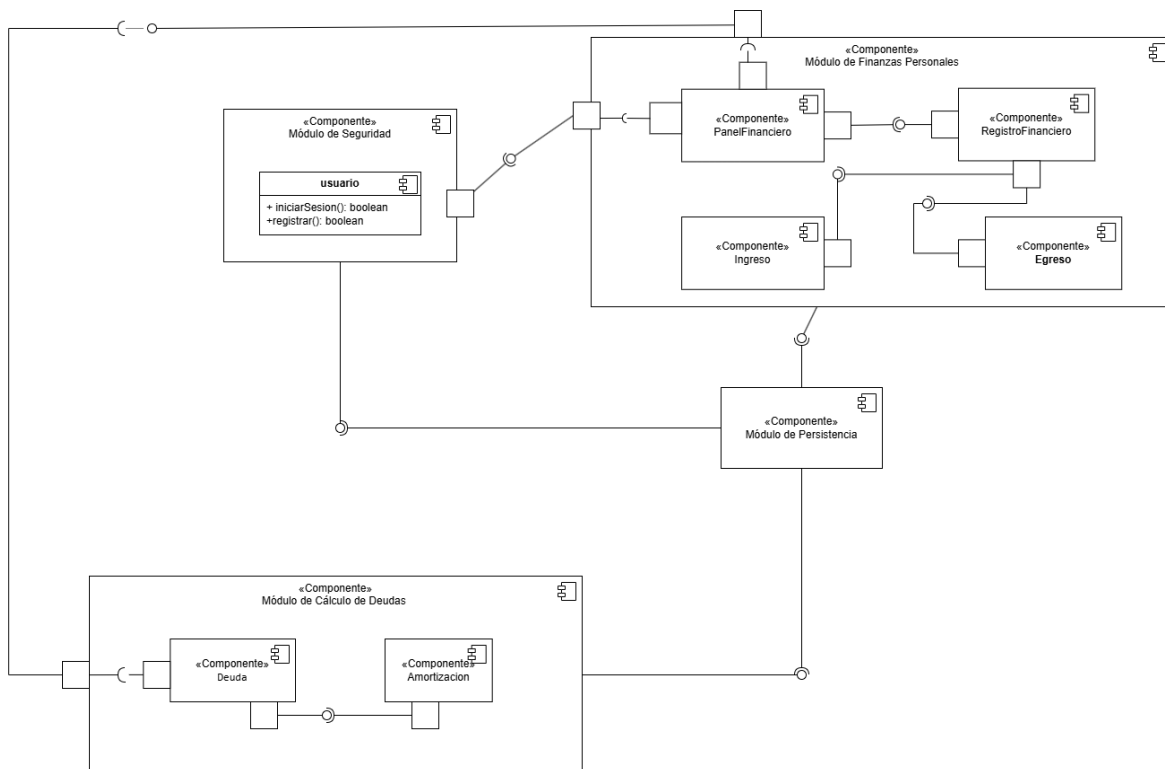
Este diagrama describe el flujo posterior:

- Formulario de egresos: ingreso y almacenamiento de datos.
- Resumen financiero: visualización de ingresos y egresos.
- Cálculo de deuda y amortización: ingreso de parámetros y generación del cuadro de amortización.

Análisis:

El diagrama refleja una interacción más compleja, con múltiples formularios y cálculos. Es fundamental garantizar que los cálculos se realicen de manera eficiente y que el sistema valide los datos ingresados. Además, se debe asegurar que el cuadro de amortización sea dinámico y fácil de interpretar.

## DIAGRAMA DE COMPONENTES

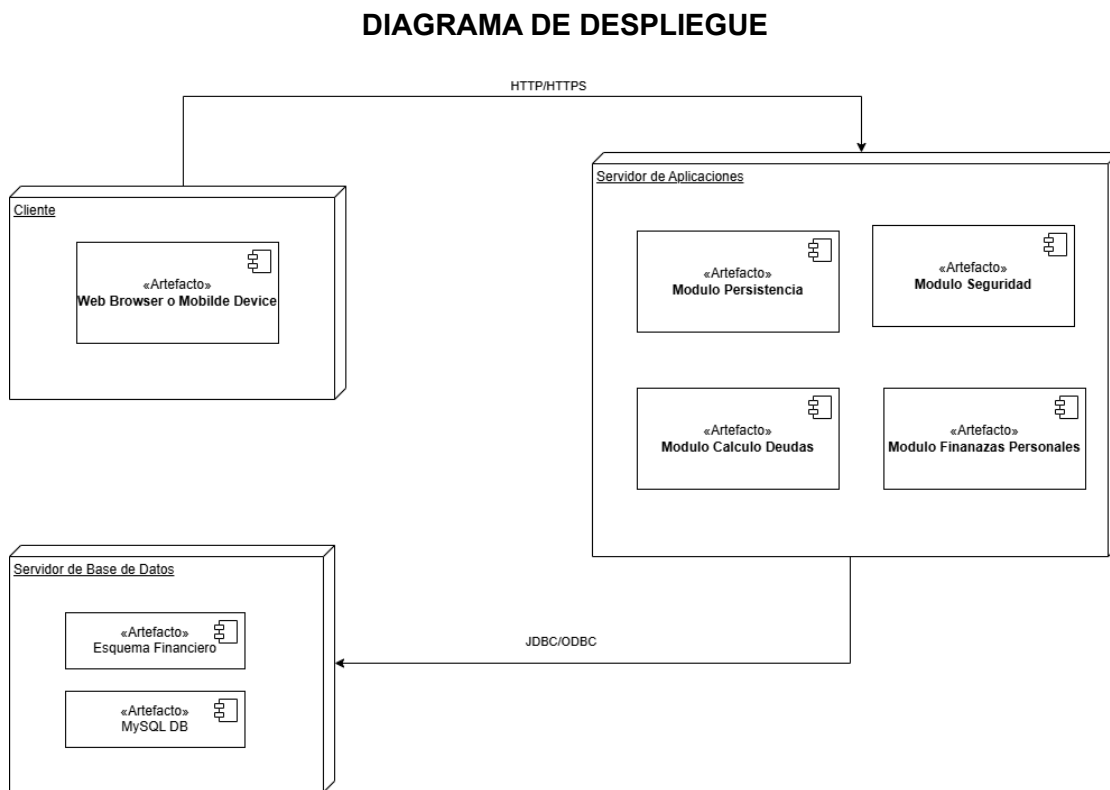


Este diagrama muestra la estructura lógica del sistema, incluyendo:

- Módulo de autenticación.
- Módulo de gestión financiera (ingresos, egresos, amortización).
- Base de datos.
- Interfaz web y móvil.

Análisis:

La separación por módulos es adecuada, lo que favorece la escalabilidad y el mantenimiento. Se recomienda definir interfaces claras entre componentes y considerar el uso de patrones de diseño como MVC para la organización del código.



Este diagrama ilustra la distribución física del sistema:

- Servidor de aplicaciones.
- Servidor de base de datos.
- Clientes (web y móvil).



**Análisis:**

El despliegue propuesto es estándar y funcional. Se debe garantizar la seguridad en la comunicación entre cliente y servidor mediante protocolos como HTTPS y considerar la implementación de copias de seguridad para la base de datos.

## OBJETIVOS

- Diseñar la arquitectura del sistema utilizando diagramas UML que representen los procesos y componentes principales.
- Definir la interacción del usuario con la aplicación mediante diagramas de secuencia que reflejen los casos de uso más relevantes.
- Establecer la estructura lógica y física del sistema a través de diagramas de componentes y despliegue.
- Garantizar la claridad y coherencia del diseño para facilitar el desarrollo posterior.

## **CONCLUSIÓN**

El diseño propuesto mediante diagramas UML proporciona una visión clara y estructurada de la arquitectura del sistema. La separación por módulos y la definición de flujos de interacción permiten una implementación eficiente y escalable. Este trabajo constituye una base sólida para el desarrollo de la aplicación, asegurando que cumpla con los requerimientos funcionales y no funcionales planteados.

## **BIBLIOGRAFÍA**

- Booch, G. (1994). Object-oriented analysis and design. Redwood City.
- Booch, G., Rumbaugh, J., Jacobson, I., Molina, J. J. G., & Martínez, J. S. (2006). El lenguaje unificado de modelado: guía del usuario. Addison-Wesley.
- Cohn, M. (2004). User stories applied: For agile software development. Addison-Wesley Professional.
- Cohn, M. (2018). User Stories and User Story Examples by Mike Cohn. [online] Mountain Goat Software.<https://www.mountaingoatsoftware.com/agile/user-stories>
- Cuevas Agustín, Gonzalo. (1991). Ingenieria del Software.Práctica de la programación, Editorial RA-MA, 1 impresión.
- Flores Cueto, J. J., & Bertolotti Zúñiga, C. (2013). Diagrama de clases en uml. 6.
- Gutiérrez, J. (s.f.). Diagramas UML de casos de uso y de requisitos.
- Larman, C. (2002). "Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process". 2nd Ed. Prentice Hall.
- Norris & Rigby. (1994). "Ingeniería de software explicada", 1 edición Editorial Megabyte-Noriega editores, México.
- Pressman, Roger. (1998). Ingeniería del Software. Un enfoque práctico, Editorial Mc. Graw-Hill, 4ta edición, 1998.
- Rumbaugh, J., Jacobson, I., & Booch, G. (2004). El Lenguaje Unificado de Modelado. Elements, 30.
- Schmuller, J. (2001). Aprendiendo UML en 24 horas. Pearson Educación.
- Schmuller, J., & Garza Marín, A. D. (2000). Aprendiendo UML en 24 horas.
- Wake, W.C. (2003). "Extreme Programming Explored" y "Refactoring Workbook"